SOFTWARE ARCHITECTURE & BACKEND READINESS REPORT

Singh Amit & Associates — CA Firm Website

A production-readiness analysis of `casinghamit.vercel.app` — a React 19 + Vite + Tailwind v4 single-page marketing site for a Chartered Accountancy firm — prepared as if scoping this codebase for real backend development: data model, API surface, admin panel, security, and a step-by-step build roadmap.

Frontend: **React 19, Vite 8, Tailwind CSS 4, Framer Motion, React Hook Form, React Router 7**

Live routes analyzed: **11** | Forms found: **3** — all frontend-only simulations | Backend present: **None**

PART 01

Project Overview

Purpose

The site is the digital storefront for **Singh Amit & Associates ("CA India")**, a Chartered Accountancy practice led by **CA Amit Singh** and based in Roorkee, Uttarakhand. It exists to generate consultation leads and phone/WhatsApp contact for tax, GST, TDS and accounting services, to establish credibility (partners, ICAI registration, awards), to publish SEO-driven tax/compliance content, and to recruit staff via a dedicated Career funnel.

Target Audience

- **Individuals & HUFs** needing income tax return filing and personal tax planning.
- **Startups, MSMEs and companies** needing GST registration/filing, TDS compliance, bookkeeping, ROC/company registration.
- **NRIs** — a distinct blog category ("NRI Taxation") signals this as a deliberately targeted segment.
- **Job seekers** (CAs, article assistants, audit/tax executives) via the Career page.
- **Referral sources / due-diligence visitors** — prospective clients checking credentials before engaging (partners, awards, certifications sections exist specifically for this).

Business Goals

- Lead generation — Contact form, fixed WhatsApp button, click-to-call links on every service page.
- Trust-building — partner bios, "25+ years / 1500+ clients" style stat banners, awards timeline, ICAI/ISO/Udyam badges.
- Content marketing / SEO — a 28-post blog with categories, tags, related posts and a newsletter capture form.
- Recruitment — job listing + application form with resume upload.

Main User Journey

Home hero (video background, "Chartered Accountancy, Elevated.") → stats banner → About-the-firm snippet + photo collage → key services grid → client testimonials carousel → embedded contact form, *or* Services mega-menu → dedicated service page (hero → sticky scroll-spy sidebar → 6–10 content sections → FAQ accordion → CTA) → `/contact`. A parallel entry path runs through the Blog for organic/SEO traffic, and a third through `/career` for recruitment.

Design Language, Palette & Typography

A consistent two-tone system repeats on every page: dark, teal-black hero sections (`bg-secondary #011818`) with a cyan accent (`highlight #00EAE7`), transitioning into white content sections that switch their accent to a deeper teal (`brand-700 #155B5C`). Buttons are rounded-`md`, uppercase, tracking-wide; the icon set is exclusively **Feather (react-icons/fi)** with one incidental Ionicons usage. Headings use **Playfair Display** (serif), body copy uses **Montserrat** — both loaded via a render-blocking `@import` in `index.css` rather than preloaded/self-hosted. Gold (`#c9a227`) marks "Specialised" service badges. The palette and hero/CTA pattern are applied with genuine discipline across all 11 routes — this is a real, if informally-documented, design system.

UI Consistency

Very high. Every hero uses the same `fadeUp/EASE=[0.22,1,0.36,1]` Framer Motion variant (re-declared per file rather than imported from one shared module — see Part 3). Every service page (GST, Income Tax, TDS, Accounting & Bookkeeping) is architecturally identical: `Hero → Sidebar (scroll-spy) → Overview/ComplianceMatters → SolutionsGrid → …6-9 content sections… → FAQSection → CTASection`, each

implemented as an *independent copy* rather than one parametrized template — a clear duplication/refactor opportunity, not a design flaw.

Responsive Behaviour

Solid. Tailwind `sm`/`lg` breakpoints drive a separate `MobileNav`, sidebars collapse into an accordion below `lg`, image collages/grids reflow to a single column, and forms stack fields with `sm:grid-cols-2` patterns throughout.

Accessibility Observations

Done well

- ✓ `aria-label` on every icon-only control (breadcrumbs, socials, WhatsApp, scroll-to-top)
- ✓ `aria-expanded` / `aria-controls` wired correctly on every FAQ accordion
- ✓ `aria-current="page"` on active breadcrumb/nav items
- ✓ Focus-visible outlines on interactive elements sitewide
- ✓ `useReducedMotion` respected in About page's animated counters/timelines

Gaps

- — Generic `<title>profile-project</title>` and **zero** `meta description` /Open Graph tags — no per-route document title management (no react-helmet equivalent)
- — `useReducedMotion` is inconsistently applied outside the About page
- — Several CTAs point at `to="/"` as a placeholder instead of being disabled/removed
- — No `robots.txt` / `sitemap.xml` found

PART 02

Complete Page Analysis

11 routes are actually reachable from `src/routes/AppRoutes.jsx`. A twelfth page tree (`src/pages/Insights/**`, backed by a separate legacy data file `blogPosts.js`) exists in the source but is not mounted in the router — it is dead code, listed at the end of this section for completeness since the brief asks not to skip anything found.

1 · Home `/`

PURPOSE	Primary landing page — establish trust fast and route visitors into services, testimonials, or the contact form.
COMPONENTS	`HomeHero`, `StatsSection`, `AboutUsSnippet`, `KeyServices`, `ClientTestimonials`, `ContactUsSection` (wraps the same `ContactForm` used on `/contact`)
SECTIONS	Video hero → stats banner (overlapping the hero, `-translate-y-50%`) → About-firm snippet with a 4-photo staggered collage → 6-card Key Services grid (links all point to `"/"` placeholder) → Swiper testimonial carousel → embedded full contact form
INTERACTIVE	Autoplaying muted background `<video>`; Swiper carousel (autoplay 3s, loop, custom prev/next, pagination dots); scroll-triggered count-up-free stat cards (static text, not animated numbers here — unlike About page's `StatCounter`)
FORMS	Contact form (see Part 5)
BUTTONS	"Get a Free Consultation" / "Know More" (both `to="/"` placeholder — not yet wired to `/contact` or an anchor)
IMAGES	Hero portrait PNG, 4 gallery photos in the About collage
LOADING STATES	None — no async data anywhere on this page

2 · About Us

`/about`

PURPOSE	Firm credibility page — story, mission/vision, partner bios, awards.
COMPONENTS	`AboutHero`, `OurStory`, `Partners` (+ `PartnerCard`, `AchievementStrip`, `StatCounter`), `MissionVision`, `AwardsRecognitions`
SECTIONS	Hero → narrative + 6-stat grid → partners (1 featured card + 3 standard cards + animated stat strip) → Mission/Vision/Impact + 3 Core Values → alternating awards timeline (2018–2023) + 4 certification badges
INTERACTIVE	`StatCounter` animates numbers counting up on scroll (respects reduced-motion); awards timeline has a scroll-progress-driven "fill" line (`useScroll` + `scaleY`)
CARDS	Featured + standard partner cards; 4 award timeline cards; 3 value cards
IMAGES	4 partner headshots (`/partners/*.webp`), hero background
USER ACTIONS	"Work With Us" → `/contact`; "Learn More" → `#our-story` anchor; partner "View Profile"/LinkedIn/email links are all # placeholders

FINDING

Three different "years of experience" numbers appear across this single page/site: **12+ years** (OurStory copy), **25+ years** (AchievementStrip's `firmStats` data), **20+ years** (Home hero copy). A content-consistency bug worth fixing regardless of backend work, and a good argument for centralizing firm stats in one database table (see Part 6, `firm_stats`).

3 · GST Services

`/services/gst-services`

PURPOSE	Deep-dive service page for GST registration, filing, ITC and compliance.
COMPONENTS	`Hero`, `Sidebar` (scroll-spy, 10 links), `ComplianceMatters`, `GSTSolutionsGrid`, `FilingCalendar`, `ITC`, `Industries`, `WhyChooseUs`, `ComplianceHealthCheck`, `WorkingProcess`, `FAQSection`, `CTASection`
SECTIONS	12 anchor-linked sections; sticky sidebar (desktop) / collapsible accordion nav (mobile) tracks scroll position via a custom `useScrollSpy` hook
INTERACTIVE	FAQ accordion (6 Q&A, single-open); scroll-spy sidebar; two animated "process" rails driven by `useScroll`
TABLES	None literally, but `FilingCalendar` renders a static GSTR-1/3B/9/9C reference "table" as a card grid, not an actual data table or interactive calendar
CARDS	6 solutions cards, 6 compliance-check items, 8 industry cards, 5 ITC cards, 6 "why us" cards, 7 process-step cards
USER ACTIONS	"Get GST Consultation" → `/contact`; "Contact GST Expert" → `tel:+911204000350`

Despite the names, **"Filing Calendar" and "Compliance Health Check" are static informational layouts**, not an interactive due-date calendar or a real assessment tool — worth flagging since both are strong candidates for genuinely interactive (and backend-worthy) upgrades later.

4 · Income Tax & Tax Advisory

`/services/income-tax-advisory`

PURPOSE	Deep-dive page for ITR filing, tax planning/advisory/compliance, notices & assessment, business tax.
COMPONENTS	`Hero`, `Sidebar` (8 links), `Overview`, `ITRFiling`, `TaxPlanning`, `TaxAdvisory`, `TaxCompliance`, `NoticeAssessment`, `BusinessTaxConsultation`, `ProcessTimeline`, `FAQSection`, `CTASection`
SECTIONS	10 anchor sections; same sidebar/scroll-spy pattern as GST
INTERACTIVE	FAQ accordion (6 Q&A referencing specific sections/thresholds — ₹5,000 late fee u/s 234F, ₹3.75L deduction figures); animated process rail
CARDS	4 tax-advisory items, 4 compliance items, 4 planning items, 6 business-type tiles, 7 process steps
USER ACTIONS	"Get Tax Consultation" → `/contact`; "Talk to an Expert" → `tel:+911204000350`

FINDING

A `WhyChooseUs.jsx` component exists in this page's folder, fully written, but is **never imported/rendered** — dead code. Its sidebar (8 entries) also has no "FAQs" or "Contact" link even though those sections exist on the page — a minor nav/content mismatch versus GST's 10-entry sidebar.

5 · TDS Compliance

`/services/tds-compliance`

PURPOSE	Deep-dive service page for TDS deduction, filing and reconciliation.
COMPONENTS	Same architecture family as GST: `Hero, Sidebar, ComplianceMatters, TDSSolutionsGrid, FormsReconciliation, WhoNeedsTDS, WhyChooseUs, ComplianceHealthCheck, WorkingProcess, FAQSection, CTASection`
NOTABLE DIFFERENCE	`FormsReconciliation` and `WhoNeedsTDS` replace GST's `ITC/Industries` — the TDS-specific "which form (24Q/26Q/27Q) applies to me" and "who must deduct TDS" content
USER ACTIONS	Identical CTA/anchor pattern to GST and Income Tax pages

6 · Accounting & Bookkeeping

`/services/accounting-bookkeeping`

PURPOSE	Deep-dive service page for bookkeeping, MIS/reporting and accounting support.
COMPONENTS	Same family again: `Hero, Sidebar, Overview, ServicesGrid, Industries, Benefits, WhyChooseUs, WorkingProcess, FAQSection, CTASection`
NOTABLE DIFFERENCE	`Benefits` is unique to this page (vs. a "compliance matters"/"ITC" style section on the other three)

Four service pages, four independent component trees, one identical structural pattern (hero → sidebar → 6-9 sections → FAQ → CTA). This is the single biggest React refactor opportunity on the site (see Part 3).

7 · Life@SAA

`/life-at-saa`

PURPOSE	Culture/employer-branding page supporting recruitment.
COMPONENTS	`LifeHero`, `OurCulture`, `LifeGallery`, `InsightsArticles`, `MomentsGallery`, `GrowCTA`
SECTIONS	Hero (no CTA buttons, unlike other heroes) → 3 culture-pillar cards → 4-card hover-reveal gallery → 3-card "Insights" teaser → 8-photo asymmetric mosaic → closing CTA
IMAGES	8 distinct photos in the mosaic; gallery recycles the same 3 background images used elsewhere on the site (placeholder imagery, not final photography)
USER ACTIONS	"Explore Careers" → `/career` (functional)

BROKEN LINK

All three `InsightsArticles` "Read More" links point to `to="/"` — a hardcoded blog-teaser grid with fake dates that doesn't actually link to any article. This is the clearest signal that a real CMS-backed article feed was intended here but never wired up.

8 · Career

`/career`

PURPOSE	Recruitment page — job listing + application form.
COMPONENTS	`CareerHero`, `CurrentOpenings`, `WhyJoinSAA`, `ApplyNow`, `CareerFAQ`
STATE	`Career.jsx` holds `selectedPosition`; clicking a job's "Apply Now" scrolls to and pre-fills the application form's position dropdown — the only cross-component interaction found on any of the 11 pages
FORMS	Job application form (see Part 5) — **the single clearest backend requirement on the whole site**
CARDS	5 job-opening cards, 4 "why join" benefit cards
INTERACTIVE	FAQ accordion (4 Q&A); file-upload dropzone (resume, 5MB client-side cap)

9 · Contact `/contact`

PURPOSE	Primary conversion page.
COMPONENTS	`ContactHero`, `ContactForm` (+ `ContactInfoCards`, `FloatingSocialBar`), `WhyContactUs`, `LocationMap`
FORMS	Full enquiry form — 9 fields (see Part 5)
INTERACTIVE	Conditional "Other service" text field (appears/animates in only when "Other" is selected); embedded map

10 · Blog `/blogs`

PURPOSE	SEO content hub — 28 posts across 10 categories.
COMPONENTS	`BlogHero`, `FeaturedArticle`, `LatestArticles` (+ `BlogCard`), `BlogSidebar`, `Pagination`, `NewsletterSection`
INTERACTIVE	Live client-side search (no debounce, filters on every keystroke), category filter with live counts, "popular tags" that feed the same search box, pagination (6 posts/page, ellipsis-compressed page list)
FORMS	Newsletter email signup (see Part 5)
STATE HANDOFF	Arrives here via `useLocation().state` from `BlogDetails`' sidebar (clicking a tag/category on an article page routes here pre-filtered)

11 · Blog Post Detail

`/blog/:slug`

PURPOSE	Individual article page, dynamic by `slug` route param.
COMPONENTS	`ArticleHero`, `ArticleMeta`, `ArticleContent`, `KeyTakeaways`, `ArticleFAQs`, `AuthorCard`, `PostNavigation`, `RelatedArticles`, `BlogSidebar` (reused)
CONTENT RENDERING	`ArticleContent` is a hand-rolled lightweight parser (not a markdown library) recognising `## / ### / > / - / 1. / !!! / **bold**` — produces real semantic HTML without `dangerouslySetInnerHTML`
INTERACTIVE	Per-post FAQ accordion (5 Q&A); prev/next post navigation; related-posts (same-category first, backfilled from other categories so it's never empty)
NOT FOUND STATE	Unknown slug renders a proper "Article Not Found" state with a back-to-blog link — a genuine (if client-only) 404 handling case

Dead code, not a live page

`src/pages/Insights/**` (`InsightsPage`, `ArticleDetailPage`, category filter, etc.) is a complete, self-consistent, earlier version of the blog built against a different data file (`src/data/blogPosts.js`). Neither is referenced by `AppRoutes.jsx` — confirmed unreachable. Recommend deleting both once the current Blog/BlogDetails implementation is confirmed as the one moving forward, to stop future contributors from editing the wrong copy.

PART 03

Component Architecture

Global / Shared Shell

COMPONENT	PURPOSE	PARENT	PROPS	STATE
`Layout`	App shell: header, routed outlet, footer, floating actions	Router root	—	none
`Header/DesktopNav/MobileNav`	Site navigation, services mega-menu	Layout	—	menu open/close, scroll-elevated style (`useScrollPositi`
`Footer`	Sitemap links, contact info, socials	Layout	—	none
`FloatingActions`	WhatsApp button + scroll-progress "back to top" ring	Layout	—	scroll %, visibility
`FloatingSocialBar`	Fixed social icon rail, reveals once past 80vh, stays revealed	ContactForm (embedded on Home + Contact)	—	`isVisible` (IntersectionObserv fires once)
`Breadcrumb`	Shared "Home / Section / Page" trail for dark hero sections	Every Hero	`items[]`, `delay`, `topClass`	none
`Container`	Max-width/gutter wrapper	Everywhere	`as`, `className`	none

Duplicated-but-should-be-shared: the "Service Page" family

GST Services, Income Tax Advisory, TDS Compliance and Accounting & Bookkeeping each ship their *own copy* of an architecturally identical pattern. The refactor target:

PROPOSED SHARED COMPONENT	REPLACES	WOULD TAKE AS PROPS
`ServiceHero`	4× `Hero.jsx`	`title, breadcrumbItems, description, primaryCta, secondaryCta, bgImage`
`ServiceSidebar`	4× `Sidebar.jsx` + `sectionsConfig.js`	`sections: {id, label, icon}[]`
`ServiceFAQAccordion`	4× `FAQSection.jsx`	`faqs: {question, answer}[]`
`ServiceCTA`	4× `CTASection.jsx`	`heading, body, primaryCta, secondaryCta`
`useScrollSpy(ids)`	already shared ✅ — good existing example to model the rest on	—

Full Component Hierarchy

```
<RouterProvider> Layout ├─ ScrollToTop (resets scroll on route change) ├─ Header → DesktopNav, MobileNav ├─ <Outlet> │ ├─ / → Home │ │ └─ HomeHero, StatsSection, AboutUsSnippet, KeyServices, ClientTestimonials, │ │ ContactUsSection → ContactForm → ContactInfoCards, FloatingSocialBar │ ├─ /about → About │ │ └─ AboutHero, OurStory, Partners → PartnerCard, AchievementStrip → StatCounter, │ │ MissionVision, AwardsRecognitions │ ├─ /contact → ContactPage │ │ └─ ContactHero, ContactForm(shared), WhyContactUs, LocationMap │ ├─ /services/income-tax-advisory → IncomeTaxAdvisory │ │ └─ Hero, Sidebar, Overview, ITRFiling, TaxPlanning, TaxAdvisory, TaxCompliance, │ │ NoticeAssessment, BusinessTaxConsultation, ProcessTimeline, FAQSection, CTASection │ │ (WhyChooseUs.jsx exists, unused) │ ├─ /services/gst-services → GSTServices │ │ └─ Hero, Sidebar, ComplianceMatters, GSTSolutionsGrid, FilingCalendar, ITC, Industries, │ │ WhyChooseUs, ComplianceHealthCheck, WorkingProcess, FAQSection, CTASection │ ├─ /services/tds-compliance → TDSCompliance │ │ └─ Hero, Sidebar, ComplianceMatters, TDSSolutionsGrid, FormsReconciliation, WhoNeedsTDS, │ │ WhyChooseUs, ComplianceHealthCheck, WorkingProcess, FAQSection, CTASection │ ├─ /services/accounting-bookkeeping → AccountingBookkeeping │ │ └─ Hero, Sidebar, Overview, ServicesGrid, Industries, Benefits, WhyChooseUs, │ │ WorkingProcess, FAQSection, CTASection │ ├─ /life-at-saa → LifeAtSAA │ │ └─ LifeHero, OurCulture, LifeGallery, InsightsArticles, MomentsGallery, GrowCTA │ ├─ /career → Career (owns
```

```
selectedPosition state) | | └ CareerHero, CurrentOpenings(onApply), WhyJoinSAA,
ApplyNow(selectedPosition), CareerFAQ | ├ /blogs → BlogListing (owns
search/category/page state) | | └ BlogHero, FeaturedArticle, LatestArticles →
BlogCard, BlogSidebar, Pagination, | | NewsletterSection | └ /blog/:slug →
BlogDetails | └ ArticleHero, ArticleMeta, ArticleContent, KeyTakeaways,
ArticleFAQs, AuthorCard, | PostNavigation, RelatedArticles → BlogCard, BlogSidebar
├ Footer └ FloatingActions
```

PART 04

Backend Requirement Analysis

Every section below is currently a hardcoded JS array shipped in the bundle, or (for the three forms) a client-only simulation with **zero** server call. Ranked roughly by urgency:

1. Contact Enquiries CRITICAL

WHY	`ContactForm.onSubmit` currently does `await sleep(1200); console.log(...); reset()` — the site's primary conversion action goes nowhere. No lead is ever captured.
API	`POST /api/enquiries` (public) + `GET /api/admin/enquiries` (admin)
VALIDATION	Server must re-validate everything React Hook Form checks client-side (name ≥3, email regex, 10-digit Indian mobile, message ≥20 chars) — none of it is enforced today.
AUTH	Public write, admin-only read

2. Career Applications CRITICAL

WHY	Same simulated-submit problem, plus a résumé **file upload** with only client-side 5MB/type checks — nothing stops a malicious or oversized file server-side because there is no server.
API	`POST /api/careers/applications` (multipart)
AUTH	Public write, admin-only read/status-update

3. Newsletter Subscriptions HIGH

WHY	Same simulated pattern; needs de-duplication (unique email) and unsubscribe handling.
API	`POST /api/newsletter/subscribe`

4. Blog Content MEDIUM

WHY	28 posts hardcoded in `src/data/blog/posts.js`. Fine at this scale, but every new post currently requires a code change + redeploy. Moving to a DB unlocks an admin-editable blog and server-side search/pagination instead of shipping all 28 posts to every visitor's browser.
API	Public list/detail/related + full admin CRUD

5. Team / Partners MEDIUM

WHY

Bios, photos and social links change when partners join/leave — currently a redeploy-only edit.

6. Job Openings, Testimonials, Awards, Certifications, Firm Stats MEDIUM

WHY

All five are hardcoded arrays that a non-technical admin should be able to edit without a developer. Firm stats in particular should become *one* source of truth (see the three-different-numbers finding in Part 2).

7. SEO Metadata & Site Settings LOW–MEDIUM

WHY

No per-route `<title>`/description exists today (front-end fix, not strictly "backend"), but an admin-editable `site_settings` table (phone numbers, WhatsApp number, address, default SEO copy, social URLs) removes the need to redeploy for these routine changes.

PART 05

Forms Analysis

All three forms are built with **React Hook Form** (`mode:"onBlur"`) and share one property: `onSubmit` is `await new Promise(r=>setTimeout(r,1200))` followed by `console.log(...)` then `reset()`. **None of them call a real API.**

Form 1 — Contact Enquiry /CONTACT + EMBEDDED ON HOME

FIELD	TYPE	VALIDATION	REQUIRED
`fullName`	text	min 3 chars	Yes
`email`	email	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/`	Yes
`phone`	tel	`/^[6-9]\d{9}$/` (Indian mobile)	Yes
`company`	text	—	No
`city`	text	—	No
`service`	select	from fixed list of 10 (Loans, Consumer/Business/Tax Law, Trademark, Real Estate, Tax Prep/Advisory, Personal/Small-Business Tax, Other)	Yes
`otherService`	text	required only if `service==="Other"`, animates in/out	Conditional
`message`	textarea	min 20 chars	Yes
`privacy`	checkbox	must be checked	Yes

DATABASE TABLE	`enquiries`
API ENDPOINT	`POST /api/enquiries`
SUCCESS	`201 {id, message:"Enquiry received"}`
FAILURE	`422` field errors; `429` rate-limited; `500` generic
SPAM PROTECTION	Honeypot field + invisible reCAPTCHA v3/hCaptcha; server-side rate limit per IP
SANITISATION	Strip HTML/scripts from every text field before storage; normalise phone/email
RATE LIMITING	5 requests / 10 min / IP

Form 2 — Career Application /CAREER

FIELD	TYPE	VALIDATION	REQUIRED
`applicantName`	text	min 3 chars	Yes
`applicantEmail`	email	email regex	Yes
`applicantPhone`	tel	Indian mobile regex	Yes
`position`	select	one of 5 job titles + "General Application"; pre-filled from job card click	Yes
`resume`	file	`.pdf,.doc,.docx`, ≤5MB (client-only today)	Yes

DATABASE TABLE	`job_applications`
API ENDPOINT	`POST /api/careers/applications` (multipart/form-data)
SUCCESS	`201 {id}`, triggers HR notification email
FAILURE	`422` validation, `413` file too large, `415` unsupported file type
SPAM PROTECTION	Honeypot + rate limit; optional email-verification link before the file is retained long-term
SANITISATION	Re-check MIME type server-side (never trust the extension); randomise stored filename; strip EXIF/metadata
RATE LIMITING	3 requests / hour / IP

Form 3 — Newsletter Signup /BLOGS

FIELD	TYPE	VALIDATION	REQUIRED
`email`	email	email regex	Yes

DATABASE TABLE	`newsletter_subscribers`
API ENDPOINT	`POST /api/newsletter/subscribe`
SUCCESS	`201` or `200` if already subscribed (idempotent)
FAILURE	`422` invalid email, `429` rate-limited
SPAM PROTECTION	Double opt-in confirmation email recommended
RATE LIMITING	5 requests / hour / IP

Pseudo-form — Blog Search & Category Filter NO BACKEND NEEDED AS-IS

`BlogSidebar`'s search input filters the 28 already-downloaded posts entirely client-side, live on every keystroke, no submit action. It only becomes a "form" worth an endpoint once posts move server-side (Part 4, item 4) and the dataset grows beyond what's reasonable to ship whole to the client.

PART 06

MySQL Database Design

22 tables, InnoDB, `utf8mb4`. Grouped by domain.

Auth & Settings

TABLE	KEY COLUMNS	KEYS / CONSTRAINTS
`admins`	`id` PK, `name`, `email` UNIQUE, `password_hash`, `role` ENUM(admin,editor), `created_at`, `last_login_at`	UNIQUE(email)
`site_settings`	`id` PK, `setting_key` UNIQUE, `setting_value` TEXT, `updated_at`	UNIQUE(setting_key)
`media`	`id` PK, `file_name`, `file_path`, `file_type`, `uploaded_by` FK→admins.id, `alt_text`, `created_at`	FK(uploaded_by)

Lead Capture

TABLE	KEY COLUMNS	KEYS / CONSTRAINTS
`enquiries`	`id` PK, `full_name`, `email`, `phone`, `company` NULL, `city` NULL, `service_requested`, `other_service` NULL, `message` TEXT, `status` ENUM(new,contacted,closed) DEFAULT new, `source_page`, `created_at`	INDEX(status), INDEX(created_at)
`job_applications`	`id` PK, `applicant_name`, `applicant_email`, `applicant_phone`, `position_id` FK→job_openings.id NULL, `position_title_snapshot`, `resume_path`, `status` ENUM(new,shortlisted,rejected,hired) DEFAULT new, `notes` TEXT NULL, `created_at`	FK(position_id) ON DELETE SET NULL
`newsletter_subscribers`	`id` PK, `email` UNIQUE, `status` ENUM(subscribed,unsubscribed), `subscribed_at`, `unsubscribed_at` NULL	UNIQUE(email)

Blog

TABLE	KEY COLUMNS
blog_categories	id PK, name UNIQUE, slug UNIQUE
blog_tags	id PK, name UNIQUE, slug UNIQUE
blog_authors	id PK, name, role, avatar_path, bio, linkedin_url, email
blog_posts	id PK, slug UNIQUE, title, category_id FK, author_id FK, published_date DATE, reading_time_minutes, featured_image_path, is_featured BOOL, summary, content LONGTEXT, status ENUM(draft,published), views_count DEFAULT 0, created_at, updated_at
blog_post_tags	post_id FK, tag_id FK
blog_key_takeaways	id PK, post_id FK, sort_order, point_text
blog_faqs	id PK, post_id FK, sort_order, question, answer TEXT

Firm Content

TABLE	KEY COLUMNS	K
team_members	id PK, name, designation, qualifications JSON, experience_label, bio, photo_path, expertise JSON, linkedin_url, email, display_order, is_active BOOL	IN
testimonials	id PK, client_name, client_handle, review_text, avatar_path, rating TINYINT, is_featured BOOL, display_order, created_at	—
job_openings	id PK, title, description, experience_required, location, type ENUM(Full-Time,Part-Time,Articleship), icon, is_active BOOL, display_order, created_at	IN
awards	id PK, year, title, organization, description, icon, display_order	—
certifications	id PK, name, display_order	—
firm_stats	id PK, stat_key UNIQUE, value, suffix, label, display_order	U *tr* ir
services	id PK, slug UNIQUE, name, short_description, hero_heading, icon, is_placeholder BOOL, display_order	U y
service_faqs	id PK, service_id FK, question, answer, sort_order	FI

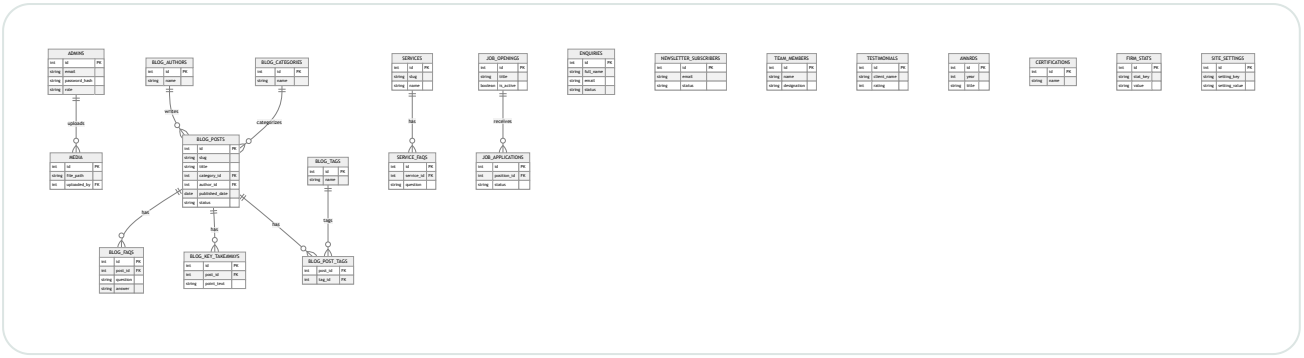
Sample Records

```
-- enquiries
INSERT INTO enquiries (full_name, email, phone, service_requested, message, status, source
VALUES ('Ravi Mehta', 'ravi@example.com', '9876543210', 'Tax Advisory',
        'Need help planning FY26 advance tax.', 'new', '/contact');

-- blog_posts (abridged)
INSERT INTO blog_posts (slug, title, category_id, author_id, published_date, reading_time_
VALUES ('essential-income-tax-planning-strategies-individuals-hufs',
        'Essential Income Tax Planning Strategies for Individuals & HUFs',
        1, 1, '2026-07-10', 8, 1,
        'Effective tax planning helps you legally save taxes...', 'published');

-- job_applications
INSERT INTO job_applications (applicant_name, applicant_email, applicant_phone, position_
VALUES ('Ananya Rao', 'ananya@example.com', '9812345670', 1, 'Chartered Accountant', '/up
```

ER Diagram



`enquiries`, `newsletter_subscribers`, `team_members`, `testimonials`, `awards`, `certifications`, `firm_stats` and `site_settings` are intentionally standalone (no incoming FKs) — they're independent content domains with no natural parent/child relationship to the blog or careers clusters, so the diagram keeps them unconnected rather than forcing an artificial join.

PART 07

REST API Design

Full detail below for the highest-value/most complex endpoints (auth, the 3 forms, blog public+admin). The remaining 13 admin content resources (`team_members, testimonials, job_openings, awards, certifications, firm_stats, services, service_faqs, blog_categories, blog_tags, blog_authors, site_settings, media`) follow one identical CRUD contract — documented once as a template immediately after, rather than repeated 13 times.

Auth

`POST` `/api/auth/login` — Public

Request: `{email, password}` · **Response 200:** `{accessToken, refreshToken, admin:{id,name,role}}` · **Errors:** 401 invalid credentials, 429 too many attempts.

`POST` `/api/auth/refresh` — Refresh token

Exchanges a valid refresh token for a new access token. **Errors:** 401 expired/invalid.

`GET` `/api/auth/me` — Bearer JWT

Returns the current admin's profile. **Errors:** 401.

`POST` `/api/auth/logout` — Bearer JWT

Revokes the refresh token server-side.

Public — Contact, Careers, Newsletter

`POST` `/api/enquiries` — Public

Request:
`{fullName, email, phone, company?, city?, service, otherService?, message, privacy}`

Response 201: `{id, status:"received"}` · **Validation:** mirrors Part 5 Form 1 exactly, server-side · **Errors:** 422, 429, 500 · Rate-limited 5/10min/IP.

`POST` `/api/careers/applications` — Public · multipart

Request (multipart):
`applicantName, applicantEmail, applicantPhone, position, resume(file)`

Response 201: `{id}` · **Errors:** 422, 413 (file >5MB), 415 (bad type), 429.

`POST` `/api/newsletter/subscribe` — Public

Request: `{email}` · **Response 200/201:** `{subscribed:true}` (idempotent) · **Errors:** 422, 429.

`GET` `/api/admin/enquiries?status=&page=` — Bearer JWT (admin)

Paginated list with status filter. **Response 200:** `{items[], total, page, pageSize}`.

`PATCH` `/api/admin/enquiries/:id` — Bearer JWT (admin)

Request: `{status}` — mark contacted/closed.

`GET` `/api/admin/careers/applications` — Bearer JWT (admin)

List/filter by position and status; includes a signed/short-lived download URL for each résumé rather than a raw public path.

Public — Blog

`GET` `/api/blog/posts?category=&q=&page=` Public

Replaces today's client-side filter/paginate. **Response 200:** `{items[], total, page, pageSize, categoryCounts}`.

`GET` `/api/blog/posts/:slug` Public

Full post incl. `keyTakeaways[]`, `faqs[]`, author, tags. **Errors:** 404 unknown slug (mirrors today's client "Article Not Found" state).

`GET` `/api/blog/posts/:slug/related` Public

Same-category-first, backfilled logic, ported from today's `getRelatedPosts()` util.

`GET` `/api/blog/categories` · `GET` `/api/blog/tags` Public

Canonical lists, replacing the static `categories.js/tags.js`.

`POST` `/api/admin/blog/posts` · `PUT` `/api/admin/blog/posts/:id` · `DELETE` `/api/admin/blog/posts/:id` Bearer JWT (editor/admin)

Full CRUD, request body mirrors the `blog_posts` schema plus nested `tags[]`, `keyTakeaways[]`, `faqs[]`.

Standard Admin CRUD Template

Applies identically to `team_members`, `testimonials`, `job_openings`, `awards`, `certifications`, `firm_stats`, `services`, `service_faqs`, `blog_categories`, `blog_tags`, `blog_authors`, `site_settings` (single-record, no delete), and `media` (upload/list/delete only):

```
GET    /api/admin/{resource}          # list, paginated
GET    /api/admin/{resource}/:id      # one record
POST   /api/admin/{resource}          # create — 201 + record
PUT    /api/admin/{resource}/:id      # full update — 200 + record
```

```
DELETE /api/admin/{resource}/:id        # 204, or 409 if referenced elsewhere

All: Authorization: Bearer <JWT>, admin or editor role
Errors: 401 no/expired token, 403 wrong role, 404 not found, 422 validation
```

Public read counterparts (`GET /api/team`, `/api/testimonials`, `/api/careers/openings`, `/api/awards`, `/api/firm-stats`, `/api/services`) expose only `is_active`/`status=published` records, no auth required.

PART 08

Authentication Analysis

Needed

- ✓ Admin login (JWT)
- ✓ Password hashing (bcrypt/argon2)
- ✓ Roles: `admin` (full access) and `editor` (content only, no user/settings management)
- ✓ Route-level permission checks on every admin endpoint

Not needed

- — Public visitor registration/login — this is a marketing site, not a client portal; nothing on the public side is gated content
- — Session-based auth — no server-rendered views to attach a session cookie to; stateless JWT fits a decoupled React SPA better

Recommended Architecture

- **Access token:** short-lived JWT (15 min), sent as `Authorization: Bearer`, holds `{sub, role}`.
- **Refresh token:** long-lived (7–30 days), stored httpOnly+Secure+SameSite=Strict cookie, rotated on each use, revocable server-side (a `refresh_tokens` table or Redis blacklist).
- **Password hashing:** bcrypt (cost ≥12) or argon2id.
- **Permissions:** a simple decorator (`@require_role("admin")`) on Flask routes rather than a full RBAC engine — two roles is enough for this site's scale.
- **Login rate limiting:** 5 attempts / 15 min / IP+email pair, exponential backoff on repeated failures.

PART 09

Admin Panel Requirements

MODULE	CREATE	READ	UPDATE	DELETE	NOTES
Blog Posts	✓	✓ list+detail	✓	✓ (soft — set status=draft first)	Rich-text image up
Blog Categories/Tags	✓	✓	✓	✓ (blocked if in use)	Simple n
Services & Service FAQs	✓	✓	✓	—	Also flips live once
Testimonials	✓	✓	✓	✓	Feature/r
Team / Partners	✓	✓	✓	✓ (soft — is_active)	Photo up editor
Job Openings	✓	✓	✓	✓ (soft — is_active)	Reorder
Job Applications	—	✓	✓ status/notes only	✓ (GDPR-style data removal)	Résumé serving
Enquiries	—	✓	✓ status only	—	Export to
Newsletter Subscribers	—	✓ + export	—	✓ unsubscribe	—
Awards / Certifications	✓	✓	✓	✓	Simple o
Firm Stats	✓	✓	✓	✓	Single so inconsist
Media	✓ upload	✓ library browser	✓ alt text	✓	Used by
Site Settings	—	✓	✓	—	Phone, W default S table
Admin Users	✓ (admin role only)	✓	✓	✓ (not self)	Invite-ba

PART 10

Folder Structure

Frontend (existing React app, extended)

```
src/
├─ api/                  # new: fetch wrappers per resource
│  ├─ client.js          # axios/fetch instance, base URL, interceptors
│  ├─ enquiries.js, blog.js, careers.js, ...
├─ components/           # existing: layout, common
├─ pages/                # existing page trees
├─ context/              # new: AdminAuthContext (admin panel only)
├─ hooks/                # existing + new: useApi, useAuth
├─ data/                 # becomes fallback/seed only once API is live
├─ utils/
└─ admin/                # new: separate admin-panel route tree
   ├─ pages/ (Dashboard, Enquiries, Blog, Team, Careers, Settings)
   └─ components/ (DataTable, Form builders, RichTextEditor)
```

Backend (Flask, new)

```
backend/
├─ app/
│  ├─ __init__.py        # app factory, extension init
│  ├─ config.py          # env-based config classes
│  ├─ extensions.py      # db, jwt, cors, limiter instances
│  ├─ models/            # SQLAlchemy models, 1 file/domain
│  ├─ schemas/            # marshmallow/pydantic request+response schemas
│  ├─ routes/             # Blueprints: auth, enquiries, careers,
│  │                        blog, team, testimonials, admin, ...
│  ├─ services/          # business logic (email, file storage, related-posts calc)
│  ├─ middleware/        # auth guard, rate limiter, error handler
│  └─ utils/             # validators, slugify, pagination helper
├─ migrations/           # Alembic
├─ uploads/               # resumes/, media/ (or swap for S3-compatible bucket)
├─ tests/
├─ .env.example
├─ wsgi.py
└─ requirements.txt
```

PART 11

Security Review

SQL INJECTION

No SQL exists yet (no backend), so risk is purely forward-looking: use SQLAlchemy's parameterised queries everywhere, especially the blog search/filter endpoint that will replace today's client-side `filterPosts()` substring match.

XSS

`ArticleContent.jsx`'s hand-rolled parser avoids `dangerouslySetInnerHTML` today — good. Once posts are admin-authored via a rich-text editor, sanitise server-side (e.g. `bleach`) *and* keep escaping on render; don't trust the editor alone.

CSRF

JWT-in-header sidesteps classic cookie CSRF for the access token, but the refresh-token cookie needs `SameSite=Strict+Secure`, and state-changing admin routes should still check an `Origin`/`Referer` header as defence-in-depth.

AUTH WEAKNESSES (CURRENT)

There is currently **no authentication anywhere** — every "backend" action is a client-side `console.log`. This is the starting point, not a regression, but it means *zero* validation, rate limiting or abuse protection exists on the live site today for its 3 forms.

VALIDATION

100% client-side (React Hook Form) today — trivially bypassed via direct HTTP requests once a real endpoint exists if server-side validation isn't mirrored exactly (see Part 5's per-form rules).

Rate Limiting

Apply per-IP limits to all three public POST endpoints (Part 5) and to `/api/auth/login`; Flask-Limiter with a Redis backend is the standard fit.

Password Security

bcrypt/argon2id hashing, minimum length + breach-list check (e.g. HaveIBeenPwned range API) on admin account creation.

FILE UPLOAD SECURITY

Résumé upload only checks size/extension *client-side* today. Server must: re-validate MIME type by content sniffing (not filename), cap at 5MB again server-side, store outside the web root with a randomised name, and ideally run an antivirus scan (e.g. ClamAV) before the file is considered "clean."

Environment Variables

None exist yet (no backend). Plan for `DATABASE_URL`, `JWT_SECRET`, `SMTP_*`, `CORS_ORIGINS`, `UPLOAD_MAX_MB`, `RECAPTCHA_SECRET` — never commit `.env`, ship a `.env.example` instead.

PART 12

Performance Review

Lazy Loading

All 11 routes are eagerly imported in `AppRoutes.jsx` — no `React.lazy()`/`Suspense` found. Every visitor downloads the Career form, the entire 28-post blog dataset, and every service page's code on first load regardless of which page they land on. Route-based code-splitting is the single biggest quick win available.

Image Optimization

Images are referenced as raw PNGs from `public/` with no responsive `srcset`/WebP/AVIF pipeline. The Home hero also autoplays a background `<video>` with no lazy-load/visibility gating — a real mobile bandwidth and LCP cost.

Component Optimization

The `EASE` constant and `fadeUp` Framer Motion variant are re-declared, verbatim, in nearly every file touched during this analysis rather than imported from one shared `motion-variants.js` — harmless at runtime, but a maintenance/bundle-dedup smell worth fixing alongside the service-page refactor in Part 3.

API / Pagination / Caching

Blog listing already paginates client-side at 6/page — carry that contract straight to the server (`?page=`) rather than reinventing it. Public GET endpoints (blog, team, testimonials) are excellent CDN/HTTP cache candidates (`Cache-Control: public, max-age=...`) since they change rarely.

Compression / SEO

Vite's build output is gzip/brotli-ready — confirm the hosting layer serves it compressed. SEO is the weakest area found: generic `<title>profile-project</title>`, no meta description, no Open Graph/Twitter cards, no `sitemap.xml`/`robots.txt`, no JSON-LD — all high-value, low-effort fixes for a locally-searched professional-services business.

Core Web Vitals

Three compounding risks: the render-blocking Google Fonts `@import` in `index.css` (should be `<link rel="preload">` or self-hosted), the autoplay hero video (LCP), and the cumulative Framer Motion + Swiper bundle weight across every page (worth a bundle-size audit once code-splitting lands).

PART 13

Backend Development Roadmap

Step 1 — Backend Setup

Flask app factory, blueprints skeleton, `Flask-SQLAlchemy`, `Flask-Migrate`, `Flask-CORS`, `Flask-JWT-Extended`, `Flask-Limiter` wired but empty.

Step 2 — MySQL Configuration

Provision a MySQL 8 instance, create the schema with `utf8mb4`, set up connection pooling (`SQLAlchemy` engine options), separate dev/staging/prod databases.

Step 3 — Models & Migrations

Author all 22 SQLAlchemy models from Part 6, run initial Alembic migration, seed lookup tables (`blog_categories`, `blog_tags`) and an initial `admins` row.

Step 4 — Authentication

Login/refresh/me/logout endpoints, password hashing, role decorator, rate-limited login.

Step 5 — Contact & Newsletter APIs

The two simplest, highest-urgency public endpoints; wire the existing `ContactForm`/`NewsletterSection` `onSubmit` handlers to real `fetch` calls, replacing the `setTimeout` simulation.

Step 6 — Career APIs

Job openings (admin CRUD + public read) and applications (multipart upload, storage, HR notification email) — the résumé pipeline is the most technically involved single piece of this roadmap.

Step 7 — Blog APIs

Migrate the 28 static posts via a one-off seed script (`src/data/blog/posts.js` → SQL inserts), build list/detail/related/category/tag endpoints, then swap the frontend's `BLOG_POSTS` import for real `fetch` calls.

Step 8 — Firm-Content APIs

Team, testimonials, awards, certifications, firm stats, services — migrate each hardcoded array the same way, resolving the years-of-experience inconsistency by making `firm_stats` authoritative everywhere it's quoted.

Step 9 — Admin Panel

A new authenticated route tree in the existing React app (or a separate app) consuming the admin endpoints, plus final security hardening pass: rate limits on every public write, CSRF/cookie flags, file-upload AV scanning, structured logging/monitoring.

Step 10 — Deployment

Containerise Flask + MySQL (Docker Compose for staging), move the frontend's API base URL to an env var, enforce HTTPS, set up CI/CD (test → migrate → deploy), configure automated DB backups.

PART 14

Final Report

1 · Completed in Frontend

- ✓ 11 fully-designed, responsive, animated pages
- ✓ Consistent hero/breadcrumb/FAQ/CTA design system
- ✓ 3 fully client-validated forms (React Hook Form)
- ✓ Client-side blog search, filter, pagination, related-posts logic
- ✓ Scroll-spy service-page navigation

2 · Needs Backend

- — All 3 forms (currently simulated, no data is ever captured)
- — Blog, team, testimonials, careers, awards, stats content (currently hardcoded arrays)
- — Admin authentication + panel (doesn't exist)
- — SEO metadata layer (per-route title/description)

3 · DB TABLES

22

Across auth, lead-capture, blog and firm-content domains.

4 · ESTIMATED APIS

~85–90

~30 documented in full detail; the rest follow the standard CRUD template in Part 7.

5 · ARCHITECTURE

Flask + SQLAlchemy + MySQL + JWT

Layered routes → services → models, matching the site's moderate real-world scale.

6–10 · Scores

OVERALL QUALITY

8/10

Polished, consistent, on-brand frontend let down only by zero backend and thin SEO.

UI / UX

8.5/10

Strong visual consistency, good accessibility hygiene, a few dead/placeholder links to clean up.

CODE ARCHITECTURE

6.5/10

Clean components, but the 4× duplicated service-page pattern and per-file `fadeUp/EASE` re-declaration is real, fixable debt.

SCALABILITY

5/10

Fine at 28 blog posts shipped to every client; won't scale past a few hundred without the backend in this report.

PRODUCTION READINESS

4/10

Every form is a no-op today — the site currently cannot capture a single lead, application, or subscriber.

Prepared as a software-architecture and backend-readiness analysis of the Singh Amit & Associates marketing site. Grounded directly in the project's own source tree (components, data files, routes) rather than the deployed build

alone, so file/line references above reflect the actual codebase at the time of writing.